

```

0000 with TEXT_IO; use TEXT_IO;
-- procedure BLOCK_SAVE is
--
-- LIST_FILE_STR: STRING(1..128); LIST_FILE_LEN: NATURAL
-- := 0; LIST_FILE: FILE_TYPE; PROCESSED_FILE_STR: STRING(1..256); PROCESSED_FILE_STR_LEN: NATURAL := 0;
--
-- on FILE_NAME_NO_PATH( PATHED_FILE_NAME: STRING ) return STRING is
--
-- begin
--   for ID in reverse PATHED_FILE_NAME'FIRST .. PATHED
--     _FILE_NAME'LAST loop
--     if PATHED_FILE_NAME( ID ) = '/' then
--       return PATHED_FILE_NAME( ID+1 .. PATHED_FILE_NAME'LAST );
--     end if;
--   end loop;
--   return PATHED_FILE_NAME;
-- end FILE_NAME_NO_PATH;
--
-- procedure BLOCK_FILE( FILE_IN, FILE_OUT: FILE_TYPE
-- ) is
--   PROCESS_IN_BUFFER: STRING(1..512); LINE_LEN: NATURAL := 0;
--   PROCESS_OUT_BUFFER: STRING(1..120); OCAR:
--   : NATURAL := 0;
--   begin
--     PROCESS_ALL_LINES: while not END_OF_FILE( FILE_IN ) loop
--       GET_LINE( FILE_IN, PROCESS_IN_BUFFER, LINE_LEN );
--       for ICAR in 1..LINE_LEN loop
--         OCAR := OCAR + 1;
--         if PROCESS_IN_BUFFER( ICAR ) = ASCII_HT then
--           PROCESS_OUT_BUFFER( OCAR ) := CHARACTER'VAL( 172 );
--         elsif CHARACTER'POS( PROCESS_IN_BUFFER( ICAR ) ) in 32..126 then
--           PROCESS_OUT_BUFFER( OCAR ) := PROCESS_IN_BUFFER( ICAR );
--         else
--           OCAR := OCAR - 1;
--         end if;
--       end if;
--     end loop;
--     PROCESS_OUT_BUFFER( OCAR ) := CHARACTER'VAL( 182 );
--     if OCAR = PROCESS_OUT_BUFFER'LAST then
--       PUT_LINE( FILE_OUT, PROCESS_OUT_BUFFER );
--       OCAR := 0;
--     end if;
--   end loop;
--   PROCESS_ALL_LINES;
--   if OCAR > 0 then
--     PUT_LINE( FILE_OUT, PROCESS_OUT_BUFFER( 1..OCAR ) );
--   end if;
--   end BLOCK_FILE;
--
-- begin
--   PUT( "NOM DU FICHER LISTE DE TEXTES SOURCES : " );
--   GET_LINE( LIST_FILE_STR, LIST_FILE_LEN );
--   OPEN( LIST_FILE, IN_FILE, LIST_FILE_STR( 1..LIST_FILE_LEN ) );
--   while not END_OF_FILE( LIST_FILE ) loop
--     GET_LINE( LIST_FILE, PROCESSED_FILE_STR, PROCESSED_FILE_STR_LEN );
--     if PROCESSED_FILE_STR_LEN /= 0 and then PROCESSED_FILE_STR( 1 ) /= '#' then
--       PROCESSED_FILE: declare
--         PROCESSED_FILE_STR: FILE_TYPE;
--         PROCESSED_FILE_STR_LEN: NATURAL;
--         PROCESSED_FILE_STR( 1..PROCESSED_FILE_STR_LEN );
--         exception when NAME_ERROR => PUT_LINE( "FICHER SOURCE INEXISTANT " & PROCESSED_FILE_STR( 1..PROCESSED_FILE_STR_LEN ) );
--         declare PATH_STRIPPED: constant STRING := FILE_NAME_NO_PATH( PROCESSED_FILE_STR( 1..PROCESSED_FILE_STR_LEN ) );
--         OUT_FILE_STR: constant STRING := "/BLK_SRC/" & PATH_STRIPPED( PATH_STRIPPED'FIRST .. PATH_STRIPPED'LAST-3 ) & "B" & PATH_STRIPPED( PATH_STRIPPED'LAST-2 .. PATH_STRIPPED'LAST );
--         begin
--           CREATE( BLOCKED_FILE, OUT_FILE_STR );
--           PUT_LINE( "FICHER SORTIE STATUS_ERROR " & OUT_FILE_STR );
--           when STATUS_ERROR => PUT_LINE( "FICHER SORTIE STATUS_ERROR " & OUT_FILE_STR );
--           CLOSE( BLOCKED_FILE );
--           CLOSE( PROCESSED_FILE );
--           exception when NAME_ERROR => PUT_LINE( "FICHER SOURCE INEXISTANT " & PROCESSED_FILE_STR( 1..PROCESSED_FILE_STR_LEN ) );
--         end PROCESSED_FILE;
--       end if;
--     end loop;
--   end BLOCK_SAVE;

```